SELF-PRUNING NEURAL NETWORK REPORT

Smaranika Dutta

1. Objective

The goal of this assignment is to design a neural network that can **learn to prune its own weights during training**, instead of applying pruning after training.

This is achieved by introducing **learnable gate parameters** that control the importance of each weigh

2. Model Design

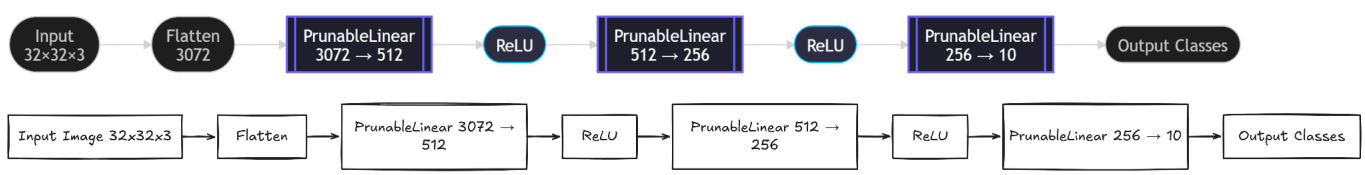
Each linear layer is replaced with a custom **PrunableLinear layer**:

$W_{eff} = W . \sigma (G)$

- W: original weights
- G: learnable gate scores
- σ(G): sigmoid → values in [0,1]

If a gate → 0, the weight is effectively pruned.

3. Architecture



The model is a feedforward neural network with integrated self-pruning using gated linear layers.

- **Input:** The input is a 32 × 32 × 3 image. It is flattened into a vector of size 3072.
- **Flatten:** The spatial input is reshaped into a 1D vector: 32 × 32 × 3 = 3072
- **Hidden Layer 1 (PrunableLinear: 3072 → 512):** A fully connected layer reduces dimensionality. Each weight is modulated by a learnable gate:$W_{eff} = W . \sigma (G)$. Here, G represents gate parameters and σ(G) maps them to [0,1]. Weights corresponding to small gate values are effectively suppressed.
- **Activation (ReLU):** Applies element-wise non-linearity:ReLU(x) = max(0, x)
- **Hidden Layer 2 (PrunableLinear: 512 → 256):** Another gated linear layer further compresses features while continuing the pruning process through learned gates.
- **Activation (ReLU):** Introduces non-linearity for deeper feature extraction.

- **Output Layer (PrunableLinear: 256 → 10):** Maps features to 10 output logits corresponding to class scores.
- **Output:** The final logits are passed to the loss function (e.g., CrossEntropy), which internally applies softmax for classification.

4. Loss Function

Total Loss=CrossEntropy+λ · Sparsity Loss

Sparsity Loss:

$$Sparsity\ Loss\ = \sum \sigma(G)$$

Why it works:

- L1 penalty pushes gates → 0
- Sigmoid ensures values ∈ [0,1]
- Many gates collapse → pruning happens

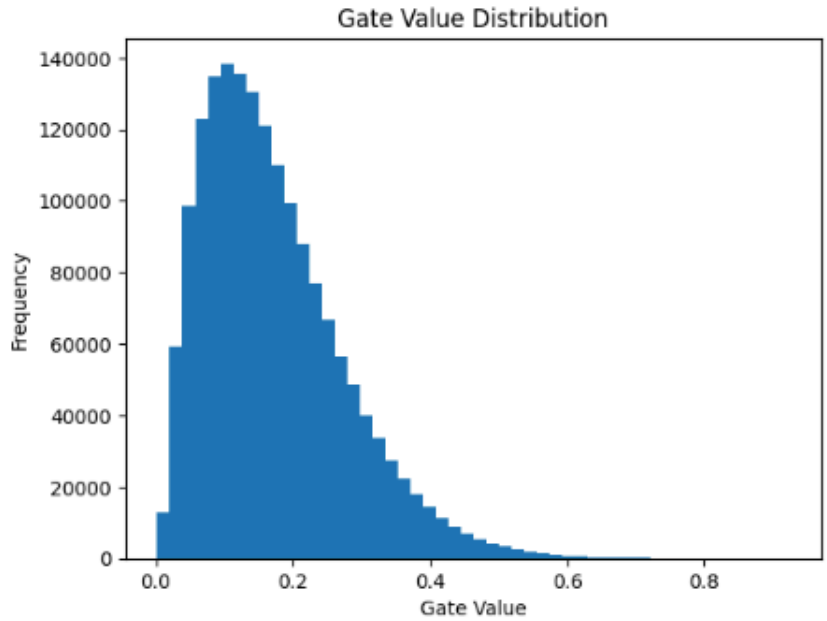
5. Experiments

Experiment 1: Early Training (5 Epochs)

Observation:

```
Final Results:
Lambda | Accuracy | Sparsity (%)
1e-05 | 54.49 | 0.04
0.0001 | 54.12 | 0.08
0.001 | 54.01 | 0.08
```

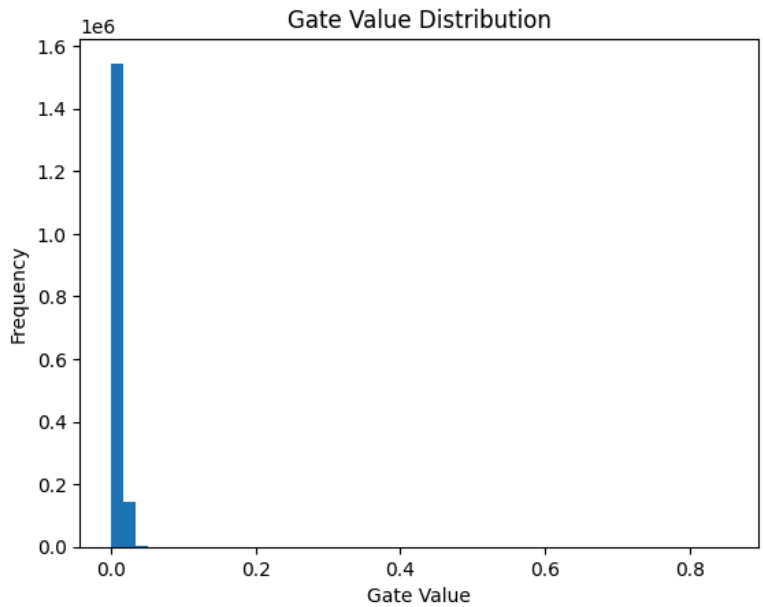
```
# Plot for last model
plot_gate_distribution(model)
```



- High accuracy
- Almost no pruning
- Sparsity loss too weak

Experiment 2: Longer Training (30 Epochs, normalized loss)

```
Final Results:
Lambda | Accuracy | Sparsity (%)
0.001 | 54.47 | 5.54
0.005 | 53.70 | 5.53
0.01 | 53.46 | 5.59
0.05 | 53.17 | 5.86
```



Observation:

- Slight increase in sparsity
- Trade-off starts appearing
- Still weak pruning

Issue Identified

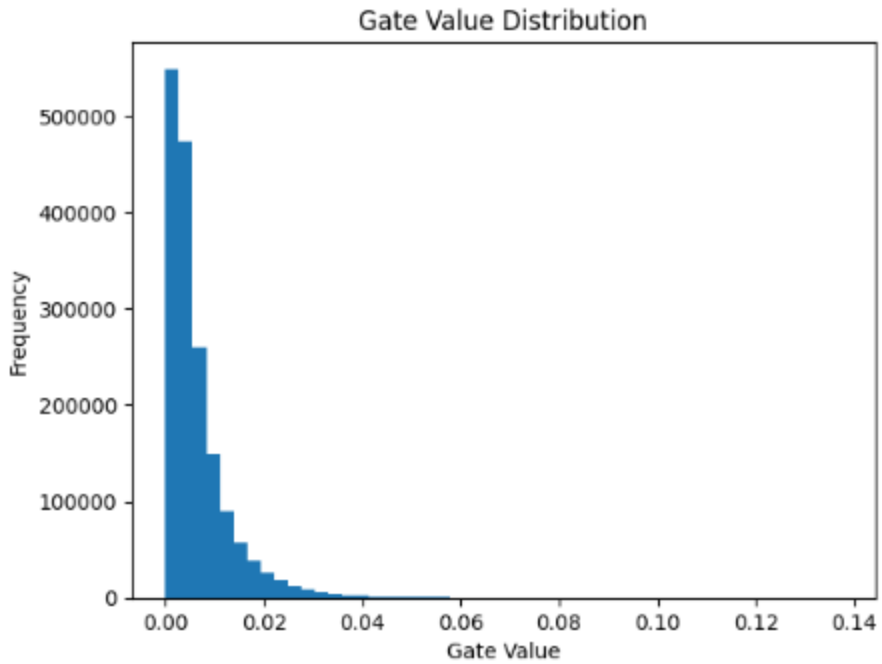
- Sparsity loss was **normalized**
- Became too small compared to classification loss

$$Classification\ Loss\ >>\ \lambda . Sparsity\ Loss$$

Result: pruning ineffective

6. Final Experiment

```
Final Results:
Lambda | Accuracy | Sparsity (%)
0.01 | 40.30 | 81.14
0.1 | 34.50 | 81.21
```



Fix Applied:

- Removed normalization of sparsity loss
- Increased λ values
- Reduced epochs for faster convergence

Final Results

Lambda	Accuracy	Sparsity (%)
0.01	40.30	81.14
0.1	34.50	81.21

7. Training Behavior

- Loss values are very high → dominated by sparsity term
- Rapid decrease across epochs
- Strong pruning pressure applied

8. Gate Distribution Analysis

- Strong spike near **0**
- Very few active gates
- Most weights effectively pruned
- This confirms:
  Successful self-pruning
  Sparse network formation

9. Trade-off Analysis

λ	Accuracy	Sparsity
Low λ	High accuracy	Low sparsity
High λ	Lower accuracy	High sparsity

Final experiment clearly shows:

- Increasing λ → **more pruning**
- But → **accuracy drops**

10. Key Insights

1. **Loss scaling is critical**
   - Small sparsity loss → no pruning
   - Large sparsity loss → aggressive pruning
2. **Sigmoid gating works effectively**
   - Enables differentiable pruning
3. **Trade-off is unavoidable**
   - Cannot maximize both accuracy & sparsity

11. Conclusion

This project successfully demonstrates a **self-pruning neural network** that learns to remove unnecessary weights during training.

- Initial experiments showed weak pruning due to improper scaling
- Final experiments achieved **~81% sparsity**, proving effectiveness
- Results validate the expected trade-off between accuracy and sparsity